

Árvore AVL

Autor: Douglas Pereira Santos
Graciano Marôto da Cunha Neto
Lara Nunes Silva

Agenda

O objetivo desta apresentação é descrever o que é, suas utilidades e a aplicação prática do indexador Árvore AVL. Trazendo sua implementação em nosso sistema de pedidos “Tia Lu Food Delivery”.

1. O que é Árvore AVL?

Uma Árvore binária de busca composta por nós, autobalanceada de forma dinâmica.

2. Rotações da Árvore AVL

Em operações de inserção ou exclusão o fator de balanceamento gera rotações simples ou duplas.

3. Aplicação prática

Aplicação da Árvore AVL no código do sistema de pedidos “Tia Lu Food App”.

4. Considerações Finais

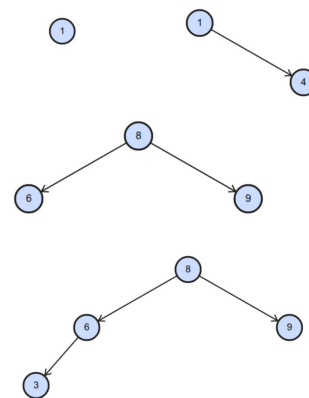
A implementação dos dados por meio de json e manipulação através do indexador Árvore AVL.

5. Referências

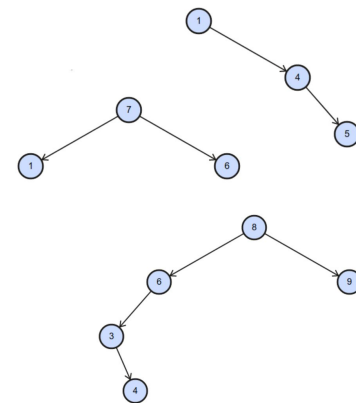
Referências bibliográficas utilizadas na pesquisa e estudo para entendimento da Árvore AVL.

O que é Árvore AVL

- Árvore binária de busca
- Autobalanceamento dinâmico
- Estrutura de dados compostas de nós
- Fator de balanço
- Complexidade de tempo de busca, inserção e exclusão



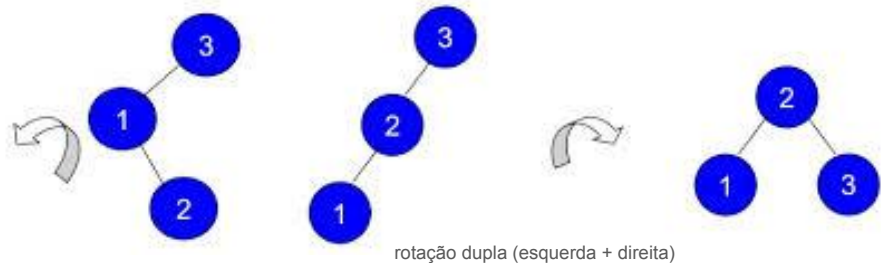
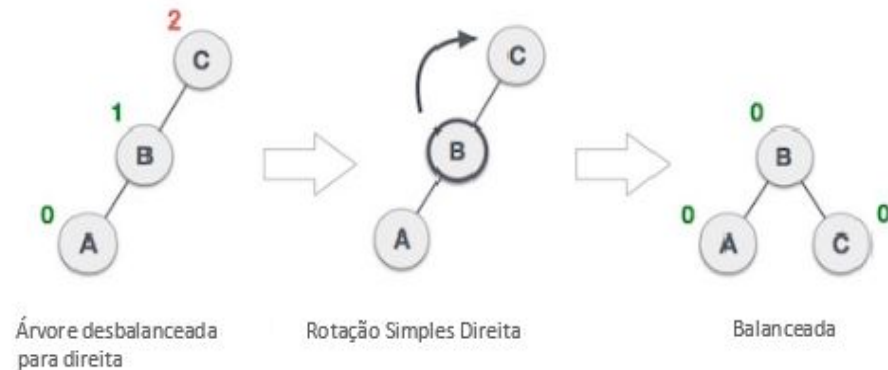
AVL



not AVL

Rotações da Árvore AVL

- Em operações, o fator de balanceamento entra em ação.
 - Inserção
 - Exclusão
- As rotações podem ser simples ou duplas
 - Direita
 - Esquerda



Aplicação no código

- Árvore AVL no código
 - Nós
 - Tamanho
 - Altura
 - Balanceamento
 - Atualizar altura

```
class Node:
    def __init__(self, key, value):
        self.key = key          # Ex: item['code']. Usado para comparação.
        self.value = value      # O dicionário completo do Item/Pedido.
        self.height = 1        # Altura do nó.
        self.left = None        # Filho esquerdo.
        self.right = None       # Filho direito.

class AVLTree:
    def __init__(self):
        self.root = None # A raiz da árvore

    def __len__(self):
        return len(self.inorder_traversal_list(self.root))

    def _get_height(self, node):
        if not node:
            return 0
        return node.height

    def _get_balance(self, node):
        if not node:
            return 0
        return self._get_height(node.left) - self._get_height(node.right)

    def _update_height(self, node):
        node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))
```

Aplicação no código

- Rotação da Árvore no código
 - rotação simples à direita
 - rotação simples à esquerda

```
# --- Rotations ---  
def _rotate_right(self, z):  
    y = z.left  
    T3 = y.right  
    y.right = z  
    z.left = T3  
    self._update_height(z)  
    self._update_height(y)  
    return y  
  
def _rotate_left(self, z):  
    y = z.right  
    T2 = y.left  
    y.left = z  
    z.right = T2  
    self._update_height(z)  
    self._update_height(y)  
    return y
```

Aplicação no código

- Utilização da Árvore no código
 - Criando função que adiciona um nó na raiz
 - Aplicando essa função para criar itens e pedidos

```
def insert(self, root, key, value):  
    if not root:  
        return Node(key, value)  
  
    if key < root.key:  
        root.left = self.insert(root.left, key, value)  
    elif key > root.key:  
        root.right = self.insert(root.right, key, value)  
    else:  
        # Chaves duplicadas (Códigos de item) não permitidas  
        return root  
  
    self._update_height(root)  
    balance = self._get_balance(root)
```

```
catalog_tree = AVLTree()  
for item in dados['catalog']:  
    catalog_tree.root = catalog_tree.insert(catalog_tree.root, item['code'], item)  
  
# Carrega Pedidos (Reconstrói a AVL - Usando a mesma chave 'code')  
orders_tree = AVLTree()  
for order in dados['all_orders']:  
    orders_tree.root = orders_tree.insert(orders_tree.root, order['code'], order)
```

Aplicação no código



- Utilização da Árvore no sistema
 - Utilizando os nós da direita e esquerda
 - Puxando todos em uma lista

```
def inorder_traversal_list(self, root):  
    """Retorna uma lista de dicionários ordenada pela chave (code)"""  
    result = []  
    if root:  
        result.extend(self.inorder_traversal_list(root.left))  
        result.append(root.value)  
        result.extend(self.inorder_traversal_list(root.right))  
    return result
```

```
def get_orders_by_status(status):  
    lista_pedidos = orders_tree.inorder_traversal_list(orders_tree.root)  
    return [o for o in lista_pedidos if o["status"] == status]
```


Considerações finais

- A implementação de dados através do **.json**
 - Simples
 - Intuitivo
 - Prático
- O indexador **Árvore AVL**
 - Organização de dados
 - Reaproveitamento de funções
 - Mais complexo

Referências

- <https://www.freecodecamp.org/portuguese/news/insercao-rotacao-e-fator-de-balanceamento-da-arvore-avl-explicados/>
- https://www.w3schools.com/dsa/dsa_data_avltrees.php
- <https://www.datacamp.com/tutorial/avl-tree>